CRYPTOLOGY ASSIGNMENT

Aditi Biju

261100690027

1. Write Python code to encrypt/decrypt – Shift Cipher

```
def encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + shift) % 26 + 97)
        else:
            result += ch
    return result
def decrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 - shift) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 - shift) % 26 + 97)
        else:
            result += ch
    return result
text = input("Enter text: ")
shift = int(input("Enter shift value: "))
encrypted = encrypt(text, shift)
print("Encrypted text:", encrypted)
```

```
decrypted = decrypt(encrypted, shift)
print("Decrypted text:", decrypted)
```

2. Caesar Cipher

```
def validate_caesar_key(key):
    if not isinstance(key, int):
        raise TypeError("Key must be an integer.")
    if key < 0 or key > 25:
        raise ValueError("Key must be between 0 and 25.")
def caesar_encrypt(text, key):
    check_text(text)
    validate_caesar_key(key)
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + key) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + key) % 26 + 97)
        else:
            result += ch
    return result
def caesar_decrypt(text, key):
    check_text(text)
    validate_caesar_key(key)
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 - key) % 26 + 65)
        elif ch.islower():
```

```
            result += chr((ord(ch) - 97 - key) % 26 + 97)
        else:
            result += ch
    return result
text = input("Enter text: ")
key = int(input("Enter key (0-25): "))
encrypted = caesar_encrypt(text, key)
print("Encrypted:", encrypted)
print("Decrypted:", caesar_decrypt(encrypted, key))
```

3. ROT13

```
def rot13(text):
    check_text(text)
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + 13) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + 13) % 26 + 97)
        else:
            result += ch
    return result
text = input("Enter text: ")
encrypted = rot13(text)
print("Encrypted:", encrypted)
print("Decrypted:", rot13(encrypted))
```

4. Atbash Cipher

```
def atbash(text):
    check_text(text)
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr(90 - (ord(ch) - 65))
        elif ch.islower():
            result += chr(122 - (ord(ch) - 97))
        else:
            result += ch
    return result
text = input("Enter text: ")
encrypted = atbash(text)
print("Encrypted:", encrypted)
print("Decrypted:", atbash(encrypted))
```

5. Affine Cipher

```
import math
def validate_affine_keys(a, b):
    if not isinstance(a, int) or not isinstance(b, int):
        raise TypeError("Keys must be integers.")
    if math.gcd(a, 26) != 1:
        raise ValueError("Key 'a' must be relatively prime to 26.")
    return a % 26, b % 26
def find_inverse(a):
    for i in range(26):
        if (a * i) % 26 == 1:
            return i
```

```
    raise ValueError("No inverse exists for this value.")
def affine_encrypt(text, a, b):
    check_text(text)
    a, b = validate_affine_keys(a, b)
    result = ""
    for ch in text:
        if ch.isupper():
            x = ord(ch) - 65
            result += chr((a * x + b) % 26 + 65)
        elif ch.islower():
            x = ord(ch) - 97
            result += chr((a * x + b) % 26 + 97)
        else:
            result += ch
    return result
def affine_decrypt(text, a, b):
    check_text(text)
    a, b = validate_affine_keys(a, b)
    inverse = find_inverse(a)
    result = ""
    for ch in text:
        if ch.isupper():
            y = ord(ch) - 65
            result += chr((inverse * (y - b)) % 26 + 65)
        elif ch.islower():
            y = ord(ch) - 97
            result += chr((inverse * (y - b)) % 26 + 97)
        else:
            result += ch
```

```
    return result
text = input("Enter text: ")
a = int(input("Enter key a: "))
b = int(input("Enter key b: "))
encrypted = affine_encrypt(text, a, b)
print("Encrypted:", encrypted)
print("Decrypted:", affine_decrypt(encrypted, a, b))
```